SkillNavora

MODULE 5 HANDBOOK

# Module 5: RAG and Enterprise Knowledge Systems

6 sections, every topic explained with how it works, examples and resources.

**From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.**

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

# Module 5: RAG and Enterprise Knowledge Systems

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

### Sections


1. 5.1 RAG Fundamentals
2. 5.2 Document Processing
3. 5.3 Chunking and Embeddings
4. 5.4 Vector Databases
5. 5.5 Advanced RAG
6. 5.6 RAG Evaluation


MODULE 5 · SECTION 5.1

## 5.1 RAG Fundamentals

#### DEFINITION

> Retrieval-augmented generation (RAG) finds relevant passages from your documents at question time and gives them to the model, so answers are grounded in current, private sources.

#### HOW IT WORKS

- Indexing: documents are split into chunks, embedded, and stored with metadata.
- Retrieval: the question is embedded and the nearest chunks are returned.
- Generation: the model answers using only those chunks and cites them.
- RAG is preferred over fine-tuning when facts change often or must be traced to sources.

#### EVERY TOPIC EXPLAINED

**What retrieval-augmented generation is and why it is needed.** Retrieval-augmented generation (RAG) retrieves relevant information from your own sources and adds it to the prompt so the model answers from that information. It is needed because models do not know your private or recent data, and RAG reduces hallucination and allows citations.

**RAG vs fine-tuning; architecture; retrieval and generation; grounding.** RAG vs fine-tuning: RAG updates knowledge instantly by updating documents; fine-tuning changes behaviour. Architecture: ingest documents → chunk → embed → store; at question time: embed query → retrieve top chunks → build prompt → generate → cite. Grounding means the answer is restricted to, and traceable to, the retrieved sources.

#### WORKED EXAMPLE

> An HR bot answers leave-policy questions by retrieving the current policy section and quoting it, with a link to the source document.

#### RESOURCES

- Pinecone: RAG chatbot guide https://docs.pinecone.io/guides/get-started/build-a-rag-chatbot

- Anthropic: contextual retrieval https://www.anthropic.com/news/contextual-retrieval

# 5.2 Document Processing

## DEFINITION

> Document processing turns PDFs, Word files, spreadsheets and web pages into clean text and structure that a RAG system can index.

## HOW IT WORKS

- Text PDFs are parsed directly; scanned PDFs need OCR. Tables need special handling so rows stay together.
- Metadata (title, date, author, section, source URL) is extracted and stored so answers can be filtered and cited.
- Classification tags each document (policy, invoice, contract) so retrieval can scope searches.

## EVERY TOPIC EXPLAINED

**PDF, Word, Excel and HTML processing; web page extraction.** PDF: extract text with PyMuPDF or pdfplumber; Word: python-docx; Excel: pandas/openpyxl; HTML and web pages: BeautifulSoup or a crawler, removing menus and footers. Keep headings and tables, which carry meaning.

**OCR, document parsing, metadata extraction, document classification.** OCR (Tesseract, cloud document AI) reads text from scanned images. Document parsing finds structure (titles, sections, tables, lists). Metadata extraction captures title, author, date, source and section. Document classification labels type (invoice, contract, policy) so retrieval can filter by it.

## WORKED EXAMPLE

> A 200-page PDF is parsed page by page; headings become section metadata, and tables are exported as rows of text.

## RESOURCES

- PyMuPDF documentation https://pymupdf.readthedocs.io/en/latest/
- Tesseract OCR docs https://tesseract-ocr.github.io/

# 5.3 Chunking and Embeddings

## DEFINITION

> Chunking splits documents into pieces small enough to retrieve and fit in context, and embedding turns each piece into a vector.

## HOW IT WORKS

- Fixed-size chunks (for example 500 tokens with 50 overlap) are simple; semantic chunks split at topic changes; overlap keeps sentences that span a boundary.
- Chunk size trades precision against context: small chunks match precisely but lose context; large chunks carry context but dilute the match.
- Store the embedding model name with each vector; re-embed everything if the model changes.

## EVERY TOPIC EXPLAINED

**Fixed-size, semantic and overlapping chunking; chunk size optimisation.** Fixed-size chunking cuts every N tokens; simple and predictable. Semantic chunking splits where the topic changes (headings, paragraphs). Overlapping chunking repeats some tokens between chunks so ideas are not cut in half. Optimise chunk size by testing retrieval on real questions; 300–800 tokens is a common starting range.

**Embedding generation, metadata filtering, embedding storage.** Generate embeddings for every chunk with one model, in batches. Store metadata (source, page, section, date, access group) with each chunk for filtering and citation. Store vectors in a vector database with the chunk text and an ID that links back to the source document.

## WORKED EXAMPLE

> A policy document is split at section headings, and any section over 800 tokens is split again with 80 tokens of overlap.

## RESOURCES

- Pinecone: chunking strategies https://www.pinecone.io/learn/chunking-strategies/
- LlamaIndex: node parsers https://developers.llamaindex.ai/python/framework/module_guides/loading/node_parsers/

# 5.4 Vector Databases

## DEFINITION

> A vector database stores embeddings and returns the nearest vectors to a query quickly, using approximate nearest-neighbour indexes such as HNSW.

## HOW IT WORKS

- Upsert writes vectors with IDs and metadata; query returns the top k matches, optionally filtered by metadata (for example department = HR).
- Hybrid search combines vector similarity with keyword (BM25) scores, which helps with exact names and codes.
- Options: Pinecone (managed), Qdrant (open source), pgvector (inside PostgreSQL), Supabase Vector.

## EVERY TOPIC EXPLAINED

**Pinecone, Qdrant, pgvector, Supabase Vector.** Pinecone: managed, serverless vector database. Qdrant: open source, rich filtering, self-host or cloud. pgvector: vectors inside PostgreSQL. Supabase Vector: pgvector managed by Supabase with helper APIs.

**Vector indexing, semantic search, hybrid search, metadata filtering, ranking.** Vector indexing (HNSW, IVF) makes nearest-neighbour search fast on millions of vectors. Semantic search ranks by meaning. Hybrid search combines vector and keyword (BM25) scores. Metadata filtering restricts results (department, date, permissions). Ranking orders results; a reranker can reorder the top results for better precision.

## WORKED EXAMPLE

> Query: 'parental leave for adoptive parents', filter: country = UK, top_k = 5; the matches come only from UK HR documents.

## RESOURCES

- Qdrant documentation https://qdrant.tech/documentation/
- pgvector (GitHub) https://github.com/pgvector/pgvector

# 5.5 Advanced RAG

## DEFINITION

> Advanced RAG improves retrieval quality with extra steps: query rewriting, multiple queries, reranking, hybrid search and contextual chunks.

## HOW IT WORKS

- Query rewriting turns a vague question into a clearer search; multi-query searches several phrasings and merges results.
- Reranking uses a model to reorder the top 50 candidates so the best 5 reach the answer model.
- Contextual retrieval adds a short description of each chunk's place in its document before embedding.
- Agentic RAG lets an agent decide when to search again. Knowledge graphs link entities for multi-hop questions.

## EVERY TOPIC EXPLAINED

**Hybrid retrieval, query rewriting, reranking, multi-query retrieval.** Hybrid retrieval: keyword + vector together. Query rewriting: rephrase or expand the user's question. Reranking: a cross-encoder model scores each candidate against the query. Multi-query retrieval: generate several versions of the question and merge the results.

**Contextual retrieval, agentic RAG, knowledge graphs, multimodal RAG.** Contextual retrieval: prepend a short summary of where each chunk sits in its document before embedding. Agentic RAG: an agent decides whether, where and how often to search. Knowledge graphs store entities and relations for multi-hop questions. Multimodal RAG retrieves images, tables and charts as well as text.

## WORKED EXAMPLE

> The question 'what changed in Q3 pricing?' is rewritten into three searches, the results are reranked, and the top chunks answer with citations.

## RESOURCES

- Anthropic: contextual retrieval https://www.anthropic.com/news/contextual-retrieval
- LlamaIndex documentation https://docs.llamaindex.ai/

# 5.6 RAG Evaluation

## DEFINITION

> RAG evaluation measures two things: whether retrieval found the right chunks, and whether the answer is faithful to them.

## HOW IT WORKS

- Context precision: share of retrieved chunks that are relevant. Context recall: share of needed facts that were retrieved.
- Faithfulness: every claim in the answer is supported by the retrieved context. Answer relevance: the answer addresses the question.
- RAGAS is a common open-source framework for these metrics; it needs a test set of questions with reference answers. Metric names change between versions, so check the version you install.

## EVERY TOPIC EXPLAINED

**Retrieval accuracy, context relevance, faithfulness, answer relevance.** Retrieval accuracy: did the right chunks come back? Context relevance: how much retrieved text is relevant. Faithfulness: is the answer supported by context? Answer relevance: does it answer the question? Recall and precision measure finding all relevant chunks vs avoiding irrelevant ones.

**Recall and precision; retrieval latency; cost optimisation; RAGAS fundamentals; test datasets.** Track retrieval latency and cost per query (embedding + search + generation). RAGAS provides standard metrics (faithfulness, answer relevancy, context precision, context recall). Build test datasets from real user questions with reference answers and source passages, and re-run them after every change.

## WORKED EXAMPLE

> Test set of 150 questions with source passages. If context recall is 60% but faithfulness is 95%, the problem is retrieval, not generation.

## RESOURCES

- Ragas documentation https://docs.ragas.io/
- Ragas: metrics overview https://docs.ragas.io/en/stable/concepts/metrics/